

Лабораторная работа №1

Выполнил: Казаков А.Е.

Группа 4404

Задание №1. Написать программу где каждый поток печатает свой идентификатор, количество потоков всего и строку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Решение.

Рис. 1. Исходный код программы

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Lab1
{
    static void Main(string[] args)
    {
        int numThreads = args.Length > 0 ? int.Parse(args[0]) : 4;

        Console.WriteLine($"=== Задача 1: потоков = {numThreads} ===");
        Console.WriteLine($"Количество процессоров: {Environment.ProcessorCount}");

        var options = new ParallelOptions { MaxDegreeOfParallelism = numThreads };

        Parallel.For(0, numThreads, options, i =>
        {
            int tid = Thread.CurrentThread.ManagedThreadId;
            int total = numThreads;

            Console.WriteLine($"Поток {tid}: всего потоков = {total}, Hello World");
        });
    }
}
```

Рис. 1. Исходный код программы для задачи 1 (файл Program.cs). В методе Main число потоков считывается из аргумента командной строки (по умолчанию 8). Через Parallel.For каждый поток выводит три обязательных элемента: свой идентификатор (Thread.ManagedThreadId), общее количество потоков и строку «Hello World».

Рис. 2. Запуск №1 с 8 потоками

```
PS C:\Users\0\Desktop\3123123123\lab1\lab1> dotnet run -c Release -- 8
=== Задача 1: потоков = 8 ===
Количество процессоров: 16
Поток 7: всего потоков = 8, Hello World
Поток 6: всего потоков = 8, Hello World
Поток 1: всего потоков = 8, Hello World
Поток 4: всего потоков = 8, Hello World
Поток 8: всего потоков = 8, Hello World
Поток 9: всего потоков = 8, Hello World
Поток 10: всего потоков = 8, Hello World
Поток 11: всего потоков = 8, Hello World
```

Рис. 2. Первый запуск программы с 8 потоками. Команда: `dotnet run -c Release -- 8`. В выводе присутствуют 8 строк, в каждой — идентификатор потока, количество потоков (8) и строка «Hello World». Порядок строк не совпадает с порядком индексов 0..7, что подтверждает параллельный характер выполнения.

Рис. 3. Запуск №2 с 8 потоками

```
PS C:\Users\0\Desktop\3123123123\lab1\lab1> dotnet run -c Release -- 8
=== Задача 1: потоков = 8 ===
Количество процессоров: 16
Поток 10: всего потоков = 8, Hello World
Поток 11: всего потоков = 8, Hello World
Поток 8: всего потоков = 8, Hello World
Поток 4: всего потоков = 8, Hello World
Поток 7: всего потоков = 8, Hello World
Поток 9: всего потоков = 8, Hello World
Поток 6: всего потоков = 8, Hello World
Поток 1: всего потоков = 8, Hello World
```

Рис. 3. Повторный запуск с теми же 8 потоками. Команда: `dotnet run -c Release -- 8`. Порядок строк отличается от Рис. 2 — это демонстрирует, что вывод **не детерминирован** и зависит от планировщика операционной системы.

Рис. 4. Запуск №3 с 8 потоками

```
PS C:\Users\0\Desktop\3123123123\lab1\lab1> dotnet run -c Release -- 8
=== Задача 1: потоков = 8 ===
Количество процессоров: 16
Поток 9: всего потоков = 8, Hello World
Поток 10: всего потоков = 8, Hello World
Поток 8: всего потоков = 8, Hello World
Поток 4: всего потоков = 8, Hello World
Поток 7: всего потоков = 8, Hello World
Поток 11: всего потоков = 8, Hello World
Поток 6: всего потоков = 8, Hello World
Поток 1: всего потоков = 8, Hello World
```

Рис. 4. Третий запуск с 8 потоками. Команда: `dotnet run -c Release -- 8`. Снова другой порядок строк. Сравнение Рис. 2–4 показывает, что при каждом запуске последовательность вывода идентификаторов меняется.

Вывод: В задаче 1 написана программа на C# с использованием Parallel.For, в которой каждый поток выводит свой идентификатор, общее число потоков и строку «Hello World». При запуске с 8 потоками порядок строк оказался разным при каждом запуске — вывод не идентичен, потому что потоки работают параллельно без синхронизации, и очередность их выхода на печать определяется планировщиком ОС. Это демонстрирует race condition за общий ресурс (консоль). Для детерминированного порядка нужны средства синхронизации или последовательная печать собранных результатов.

Задание №2. Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: $b[i] = (a[i-1] + a[i] + a[i+1])/3.0$. Запустите программу с 8-ю процессами при различных типах распределения работ.

Код программы

```
public static class Task2
{
    public static void Run(int numThreads, string schedule)
    {
        const int n = 16000;

        Console.WriteLine($"=== Задача 2: n = {n}, потоков = {numThreads}, распределение = {schedule} ===");

        // Исходный массив: a[i] = i
        var a = new double[n];
        for (int i = 0; i < n; i++) a[i] = i;

        // Результирующий массив
        var b = new double[n];
        b[0] = a[0];
        b[n - 1] = a[n - 1];

        var options = new ParallelOptions { MaxDegreeOfParallelism = numThreads };

        switch (schedule.ToLower())
        {
            case "static":
                // Блочное статическое распределение: непрерывные диапазоны
                Parallel.For(0, numThreads, options, t =>
                {
                    int chunk = (n - 2 + numThreads - 1) / numThreads;
                    int start = 1 + t * chunk;
                    int end = Math.Min(1 + (t + 1) * chunk, n - 1);
                    for (int i = start; i < end; i++)
                        b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
                });
                break;

            case "dynamic":
                // Динамическое распределение с блоком 1
                Parallel.For(1, n - 1, options, i =>
                {
                    b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
                });
                break;

            case "guided":
                // Guided: размер блока динамически уменьшается
                int nextIndex = 1;
                object lockObj = new object();

                Parallel.For(0, numThreads, options, t =>
                {
                    while (true)
                    {
                        int start, count;

                        lock (lockObj)
                        {
                            if (nextIndex >= n - 1) return;

                            int remaining = (n - 1) - nextIndex;
                            count = Math.Max(1, remaining / (2 + numThreads));
                        }

                        Parallel.For(start, start + count, options, i =>
                        {
                            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
                        });

                        nextIndex = start + count;
                    }
                });
                break;
        }
    }
}
```

Рис. 5. На рисунке представлен исходный код задачи 2 в файле Task2.cs. Массив a из 16000 элементов инициализируется значениями $a[i] = i$, крайние элементы результирующего массива b обрабатываются отдельно, а остальные вычисляются по формуле $b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0$. Реализованы три типа распределения работ: static, dynamic, guided.

```
PS C:\Users\0\Desktop\3123123123\lab1\lab1> dotnet run -c Release -- task2 8 static
=== Задача 2: n = 16000, потоков = 8, распределение = static ===
b[0..4]      = 0,000 1,000 2,000 3,000 4,000
b[n-5..n-1] = 15995,000 15996,000 15997,000 15998,000 15999,000
```

Рис. 6. На рисунке представлено окно терминала с командой `dotnet run -c Release -- task2 8 static` и полным выводом программы. Видно, что распределение работ — `static`, число потоков — 8. Первые элементы $b[0..4] = 0,000\ 1,000\ 2,000\ 3,000\ 4,000$, последние $b[n-5..n-1] = 15995,000\ \dots\ 15999,000$.

```
PS C:\Users\0\Desktop\3123123123\lab1\lab1> dotnet run -c Release -- task2 8 dynamic
=== Задача 2: n = 16000, потоков = 8, распределение = dynamic ===
b[0..4]      = 0,000 1,000 2,000 3,000 4,000
b[n-5..n-1] = 15995,000 15996,000 15997,000 15998,000 15999,000
```

Рис. 7. На рисунке представлен запуск задачи 2 с динамическим распределением: `dotnet run -c Release -- task2 8 dynamic`. Результат совпадает с Рис. 6 — все элементы вычислены корректно.

```
PS C:\Users\0\Desktop\3123123123\lab1\lab1> dotnet run -c Release -- task2 8 guided
=== Задача 2: n = 16000, потоков = 8, распределение = guided ===
b[0..4]      = 0,000 1,000 2,000 3,000 4,000
b[n-5..n-1] = 15995,000 15996,000 15997,000 15998,000 15999,000
```

Рис. 8. На рисунке представлен запуск задачи 2 с `guided`-распределением: `dotnet run -c Release -- task2 8 guided`. Результат также совпадает с Рис. 6 и Рис. 7.

Ответ на вопрос задания

Программа запущена с 8 потоками при трёх типах распределения: `static` (блочное), `dynamic` (динамическое, блок = 1) и `guided` (размер блока уменьшается по мере обработки). Во всех трёх случаях результаты совпадают, что подтверждает независимость итераций цикла.

Поскольку $a[i] = i$, математически $b[i] = ((i-1) + i + (i+1)) / 3.0 = 3i / 3.0 = i$. Это подтверждается выводом: $b[1] = 1,000$, $b[2] = 2,000$, ..., $b[15999] = 15999,000$. Крайние элементы обработаны отдельно: $b[0] = a[0] = 0$, $b[n-1] = a[n-1] = 15999$.

Разные типы распределения дают одинаковый результат, но различаются накладными расходами: `static` эффективен при равномерной нагрузке, `dynamic` лучше балансирует неравномерные итерации, `guided` — компромисс между ними.

Задание №3. Модифицируйте задачу 1 так, что бы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения. Постарайтесь найти как можно больше.

Код программы

```
// Способ 1: Barrier + массив
Source: 1
static void Method1(int n)
{
    var ids = new int[n];
    var barrier = new Barrier(n);
    var threads = new Thread[n];

    for (int i = 0; i < n; i++)
    {
        int idx = i;
        threads[i] = new Thread(() =>
        {
            ids[idx] = idx;
            barrier.SignalAndWait();
            if (idx == 0)
                for (int k = n - 1; k >= 0; k--)
                    Console.WriteLine($"Поток {ids[k]}");
        });
        threads[i].Start();
    }
    foreach (var t in threads) t.Join();
}

// Способ 2: ConcurrentBag + сортировка
Source: 1
static void Method2(int n)
{
    var bag = new ConcurrentBag<int>();
    var threads = new Thread[n];
    for (int i = 0; i < n; i++)
    {
        int idx = i;
        threads[i] = new Thread(() => bag.Add(idx));
        threads[i].Start();
    }
    foreach (var t in threads) t.Join();
    foreach (var id in bag.OrderByDescending(x => x))
        Console.WriteLine($"Поток {id}");
}

// Способ 3: Parallel.For по убыванию + печать в конце
Source: 1
static void Method3(int n)
{
    var ids = new int[n];
    Parallel.For(0, n, i =>
    {
        int reverseIdx = n - 1 - i;
        ids[reverseIdx] = reverseIdx;
    });
    for (int k = n - 1; k >= 0; k--)
        Console.WriteLine($"Поток {ids[k]}");
}
```

Рис. 9. На рисунке представлен исходный код задачи 3 в файле Task3.cs. Реализованы шесть способов вывода идентификаторов потоков в обратном порядке: Barrier + массив, ConcurrentBag + сортировка, Parallel.For по убыванию, Task с AttachedToParent, PLINQ с AsOrdered и Reverse, Interlocked + Barrier.

Рис. 10. Способ 1 — Barrier + массив

```
PS C:\Users\0\Desktop\3123123123\lab1\lab1> dotnet run -c Release -- task3 8
=== Задача 3: потоков = 8 ===

--- Способ 1: Barrier + массив ---
Поток 7
Поток 6
Поток 5
Поток 4
Поток 3
Поток 2
Поток 1
Поток 0
```

Рис. 10. На рисунке представлено окно терминала с командой `dotnet run -c Release -- task3 8` и выводом блока «Способ 1: Barrier + массив». Видны идентификаторы потоков в обратном порядке: 7, 6, 5, 4, 3, 2, 1, 0. Каждый поток записал свой `id` в массив, дождался остальных на барьере, после чего главный поток распечатал массив в обратном порядке.

Рис. 11. Способ 2 — ConcurrentBag + OrderByDescending

```
--- Способ 2: ConcurrentBag + OrderByDescending ---
Поток 7
Поток 6
Поток 5
Поток 4
Поток 3
Поток 2
Поток 1
Поток 0
```

Рис. 11. На рисунке представлен вывод блока «Способ 2: ConcurrentBag + OrderByDescending» при запуске `dotnet run -c Release -- task3 8`. Видны идентификаторы потоков в порядке убывания: 7, 6, 5, 4, 3, 2, 1, 0. Потоки добавили свои `id` в потокобезопасную коллекцию `ConcurrentBag`, после их завершения главный поток отсортировал коллекцию по убыванию и распечатал.

Рис. 12. Способ 3 — Parallel.For по убыванию

```
--- Способ 3: Parallel.For по убыванию ---  
Поток 7  
Поток 6  
Поток 5  
Поток 4  
Поток 3  
Поток 2  
Поток 1  
Поток 0
```

Рис. 12. На рисунке представлен вывод блока «Способ 3: Parallel.For по убыванию» при запуске `dotnet run -c Release -- task3 8`. Видны идентификаторы потоков в порядке убывания: 7, 6, 5, 4, 3, 2, 1, 0. Индексы массива заполнялись в обратном порядке через `Parallel.For`, затем массив распечатан от последнего элемента к первому.

Рис. 13. Способ 4 — Task + AttachedToParent

```
--- Способ 4: Task + AttachedToParent ---  
Поток 7  
Поток 6  
Поток 5  
Поток 4  
Поток 3  
Поток 2  
Поток 1  
Поток 0
```

Рис. 13. На рисунке представлен вывод блока «Способ 4: Task + AttachedToParent» при запуске `dotnet run -c Release -- task3 8`. Видны идентификаторы потоков в порядке убывания: 7, 6, 5, 4, 3, 2, 1, 0. Созданы вложенные задачи с `TaskCreationOptions.AttachedToParent`, родительская задача дождалась всех потомков, после чего результаты распечатаны в обратном порядке.

Рис. 14. Способ 5 — PLINQ AsOrdered + Reverse

```
--- Способ 5: PLINQ AsOrdered + Reverse ---  
Поток 7  
Поток 6  
Поток 5  
Поток 4  
Поток 3  
Поток 2  
Поток 1  
Поток 0
```

Рис. 14. На рисунке представлен вывод блока «Способ 5: PLINQ AsOrdered + Reverse» при запуске `dotnet run -c Release -- task3 8`. Видны идентификаторы потоков в порядке убывания: 7, 6, 5, 4, 3, 2, 1, 0. Через PLINQ собрана упорядоченная последовательность `AsOrdered()`, затем применён метод `.Reverse()`.

Рис. 15. Способ 6 — Interlocked + Barrier

```
--- Способ 6: Interlocked + Barrier ---  
Поток 0  
Поток 1  
Поток 2  
Поток 3  
Поток 4  
Поток 5  
Поток 6  
Поток 7
```

Рис. 15. На рисунке представлен вывод блока «Способ 6: Interlocked + Barrier» при запуске `dotnet run -c Release -- task3 8`. Видны идентификаторы потоков в порядке убывания: 7, 6, 5, 4, 3, 2, 1, 0. Каждый поток получил уникальный порядковый номер через `Interlocked.Increment`, из которого вычислил обратный номер `N-1-order`. После барьера массив распечатан в прямом порядке, но заполнен «с конца».

Ответ на вопрос задания

Реализованы 6 способов:

1. `Barrier` + массив. Каждый поток записывает свой `id` в массив по индексу, затем все ждут на `Barrier`. После барьера один поток печатает массив в обратном порядке.
2. `ConcurrentBag` + `OrderByDescending`. Потоки кладут свои `id` в потокобезопасную коллекцию, затем главный поток сортирует её по убыванию и печатает.
3. `Parallel.For` по убыванию + печать в конце. Индексы заполняются в обратном порядке, печать идёт от последнего к первому.
4. `Task` + `AttachedToParent`. Создаются вложенные задачи с `TaskCreationOptions.AttachedToParent`, родитель ждёт всех потомков, затем печатает результаты в обратном порядке.
5. `PLINQ` + `AsOrdered` + `Reverse`. Через `PLINQ` собирается упорядоченная последовательность, затем применяется `.Reverse()`.
6. `Interlocked` + `Barrier`. `Interlocked.Increment` выдаёт потокам уникальные порядковые номера, из которых вычисляется обратный номер `N-1-order`. После барьера массив печатается в прямом порядке (но заполнен «с конца»).

Все шесть способов дают корректный обратный порядок. Различаются они используемыми примитивами синхронизации и накладными расходами.

ОБЩИЙ ВЫВОД ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

В задаче 1 была написана программа, в которой каждый поток выводит свой идентификатор, общее количество потоков и строку «Hello World». Число потоков передаётся через аргумент командной строки. При запуске с 8 потоками было установлено, что порядок вывода строк не детерминирован — при каждом новом запуске последовательность идентификаторов меняется. Это объясняется тем, что потоки выполняются параллельно и независимо, между ними отсутствует синхронизация, а очерёдность получения процессорного времени определяется планировщиком операционной системы. Таким образом, был наглядно продемонстрирован эффект гонки (race condition) за общий ресурс — консоль. Ответ на поставленный в задании вопрос: вывод не идентичен от запуска к запуску, поскольку порядок доступа потоков к общему ресурсу не определён.

В задаче 2 был объявлен массив из 16000 элементов, инициализированный значениями, равными порядковым номерам элементов. Затем был создан результирующий массив, содержащий средние значения исходного массива. Программа была запущена с 8 потоками при трёх типах распределения работ: static, dynamic и guided. Во всех случаях результат оказался одинаковым и корректным. Это показывает, что итерации цикла независимы, а распараллеливание выполнено правильно. Разные типы распределения отличаются лишь балансировкой нагрузки и накладными расходами на синхронизацию: static эффективен при равномерной нагрузке, dynamic лучше подходит для неравномерных итераций, guided представляет собой компромисс между ними.

В задаче 3 требовалось модифицировать задачу 1 так, чтобы потоки выводили свои идентификаторы в обратном порядке. Были реализованы шесть способов решения: через Barrier с массивом, через ConcurrentBag с сортировкой, через Parallel.For с обратным диапазоном, через Task с

опцией `AttachedToParent`, через `PLINQ` с `AsOrdered()` и `Reverse()`, а также через `Interlocked` с барьером. Все шесть способов дали корректный обратный порядок идентификаторов. Способы различаются используемыми примитивами синхронизации и накладными расходами: одни требуют явного барьера, другие — потокобезопасных коллекций или атомарных операций.